

MNIST Inference Benchmark Comparison

Performance Analysis Team

August 14, 2026

1 Introduction

This report details the implementation and results of a benchmark script designed to evaluate the inference performance of various deep learning frameworks[cite: 1]. The script compares model latency and throughput using a standard Convolutional Neural Network (CNN) architecture trained on the EMNIST dataset.

2 Methodology

2.1 Environment Settings

- The code was executed in a Python 3 environment equipped with a T4 GPU accelerator.
- The framework versions utilized include PyTorch 2.11.0+cu128, TensorFlow 2.20.0, and JAX 0.7.2.
- Additional required packages were installed during runtime, including Lightning, Flax, ONNX, ONNX Runtime, and Tabulate.

2.2 Model Architectures

The identical base CNN architecture was implemented across five different framework contexts:

- **PyTorch:** A standard `nn.Module` containing two 2D convolutional layers, max pooling, and two linear layers outputting 47 classes.
- **PyTorch Lightning:** A `LightningModule` acting as a wrapper around the base PyTorch model[cite: 1].
- **TensorFlow / Keras:** A `Sequential` model utilizing equivalent `Conv2D`, `MaxPooling2D`, and `Dense` layers.
- **JAX / Flax:** A `flax.nn.Module` leveraging `@jax.jit` to compile the inference function for high-speed execution.
- **ONNX Runtime:** An exported version of the PyTorch model run via an `InferenceSession`.

2.3 Benchmarking Procedure

The benchmark function isolates the timed inference execution[cite: 1].

- Tests were executed for batch sizes of 1 and 64[cite: 1].
- Input data structures were adapted to match the specific shape conventions of each framework (e.g., (B, C, H, W) for PyTorch and (B, H, W, C) for TensorFlow/JAX).

- The benchmarking loop consisted of a 50-iteration warmup phase to initialize the hardware, followed by a 500-iteration timed phase to calculate the average latency in milliseconds.

3 Results

The execution results reveal significant performance variations across the tested frameworks[cite: 1]. Table 1 summarizes the latency, throughput, and primary advantages recorded for each configuration.

Table 1: MNIST Inference Benchmark Comparison Table[cite: 1]

Framework	Latency (B=1)	Latency (B=64)	Throughput	Primary Advantage
ONNX Runtime	0.280 ms	3.926 ms	16,301 items/s	Graph-optimized CPU inference
JAX / Flax (JIT)	0.539 ms	0.529 ms	120,982 items/s	XLA-compiled functional stack
PyTorch Lightning	0.934 ms	26.672 ms	2,399 items/s	Clean production training workflow
PyTorch	1.214 ms	28.187 ms	2,270 items/s	Ecosystem & dynamic graph support
TensorFlow / Keras	15.491 ms	7.024 ms	9,111 items/s	Beginner-friendly API

4 Future Work: Other Explorable Frameworks

To build upon the current benchmark suite and evaluate broader production deployment scenarios, several specialized deep learning compilers, runtime engines, and inference accelerators can be incorporated:

4.1 Hardware-Accelerated Inference Engines

- **NVIDIA TensorRT:** A high-performance deep learning inference optimizer and runtime targeting NVIDIA GPUs. Incorporating TensorRT would allow evaluation of layer fusion, kernel auto-tuning, and reduced precision quantization (FP16 and INT8) on CUDA hardware.
- **Intel OpenVINO:** An open-source toolkit designed for optimizing and deploying AI models across heterogeneous Intel hardware architectures (CPUs, integrated GPUs, and NPUs). It provides deep model compression and graph transformation capabilities.

4.2 Deep Learning Compilers and Static Graphs

- **PyTorch 2.x (torch.compile):** Utilizes TorchDynamo and the Inductor compiler backend to generate optimized Triton kernels, closing the speed gap between native dynamic PyTorch execution and compiled frameworks like JAX.
- **Apache TVM:** An end-to-end open-source machine learning compiler framework that automatically optimizes tensor operators across diverse hardware backends, enabling cross-platform benchmark comparisons.

4.3 Edge and Embedded Frameworks

- **TensorFlow Lite (TFLite):** A lightweight runtime tailored for mobile, edge, and embedded devices. Evaluating TFLite would provide insights into inference latency under tight memory and computational constraints.

- **Modular MAX Engine / Mojo:** A unified inference engine designed to execute heterogeneous workloads with low overhead and enhanced hardware programmability.

5 Conclusion

The benchmark demonstrates that ONNX Runtime provides the lowest latency for single-item batch sizes (0.280 ms), whereas JAX/Flax (JIT) scales exceptionally well, achieving the highest overall throughput at 120,982 items/s for a batch size of 64.

PyTorch and PyTorch Lightning prioritize ecosystem flexibility and clean code organization, though they exhibited the lowest throughput metrics in this specific test.